

Project Understanding Document

Recreating StackEval – Real-World LLM Benchmark

1. Project Overview

This project aims to recreate a simplified version of the StackEval benchmark developed by Prosus AI researchers.

Many LLM models perform similarly on academic benchmarks such as MMLU, but their performance can differ significantly when solving real developer problems.

The goal of this project is to evaluate multiple LLM models using real coding questions and human evaluation to better understand their real-world usefulness.

2. The Core Problem

Most LLM comparisons rely on academic benchmarks such as MMLU, HumanEval, or GSM8K.

These benchmarks test theoretical knowledge or synthetic tasks, which do not always reflect real developer problems.

As a result, models that perform well on academic tests may still give unhelpful answers in practical situations.

This project investigates the gap between academic benchmark performance and real-world usefulness.

3. Main Research Question

How can we build benchmarks that better predict real-world LLM performance?

Sub questions include:

- Do models that perform well on academic benchmarks also perform well on real coding problems?
- Do different models specialize in different types of tasks?
- Can cheaper or smaller models perform as well as expensive models in certain scenarios?

4. What is StackEval

StackEval is a benchmark created by Prosus researchers to evaluate LLMs using real developer questions.

Instead of relying only on automated metrics, StackEval uses human developers to evaluate model responses.

The goal is to measure whether the answers are actually helpful in real-world developer tasks.

5. Project Goal

The objective of this project is to recreate the StackEval approach on a smaller scale.

This includes collecting real coding questions, generating answers from multiple LLM models, having developers evaluate the answers, and analyzing the results.

6. Step 1 – Collect Real Coding Questions

Collect around 30–50 real coding questions from Stack Overflow.

Questions should represent real developer problems such as debugging, architecture decisions, framework usage, or performance issues.

Preferably the questions should have accepted answers to provide a reference point.

7. Step 2 – Select LLM Models

Select at least four LLM models to evaluate.

Examples include GPT-4, Claude Sonnet, Llama 3, and Gemini Flash.

The goal is to compare models from different categories such as frontier models, open-source models, and budget models.

8. Step 3 – Generate Model Answers

Each model should answer every collected question.

For example, 40 questions tested with 4 models will generate 160 answers.

All answers should be saved in a dataset for evaluation.

9. Step 4 – Human Evaluation

Recruit 2–3 software developers to evaluate the answers.

Each answer will be scored using a 0–3 rubric:

0 = completely wrong or useless

1 = partially correct but missing important details

2 = mostly correct and useful

3 = excellent answer that could be trusted.

10. Step 5 – Analyze Results

After collecting scores, analyze the performance of each model.

This includes calculating average scores, identifying strengths and weaknesses of models, and comparing results across question types.

11. Ethical Considerations

Benchmarks influence which models are adopted by companies.

Human evaluation introduces subjectivity because developers may have different preferences.

There is also a risk that models are optimized to perform well on benchmarks instead of improving real-world capability.

12. Expected Outcome

At the end of the project the result should include:

- A dataset of questions and model answers
- Human evaluation scores
- A comparison of model performance
- Insights about which models perform best for real coding tasks.

13. Skills Developed

This project helps develop skills in LLM evaluation, experimental design, benchmarking methodology, and analyzing model performance.

14. Why This Project Matters

This project demonstrates how real-world benchmarks can provide more meaningful insights than generic academic benchmarks.

It helps understand how different LLM models behave in practical developer scenarios.